

WEF1VO Web Fundamentals

Vorlesungsunterlagen

Wolfgang Hochleitner
wolfgang.hochleitner@fh-hagenberg.at

WS 2025/26

Vorlesung 2

HTML von A bis Z

In diesem Kapitel werden alle wichtigen Grundlagen erwähnt, die zur Erstellung eines HTML-Dokuments wichtig sind. Dabei geht es nicht darum, jedes einzelne Element in seinem Detail aufzuzählen, sondern vielmehr den Aufbau und die Denkweise zu vermitteln.

2.1 Kopfdaten

Vor dem eigentlichen, sichtbaren Dokumenteninhalt (der innerhalb des Elements `<body>` platziert wird), gibt es auch noch Informationen und Daten eines Dokuments, die bei jeder Seite mit angegeben werden und etwa von Suchmaschinen oder Webbrowsern ausgewertet werden. Diese sind innerhalb des `<head>`-Elements untergebracht und werden mit dem Elementen `<title>` (Seitentitel), `<base>` (Basis-URL der Seite), `<link>` (Verlinkung zu externen Ressourcen), `<style>` (direkte Angabe von Style-Informationen) und schließlich `<meta>` (Meta-Informationen) angegeben.

2.1.1 Seitentitel

Wie bereits in Vorlesung 1 kurz erwähnt, repräsentiert das `<title>`-Element den *Titel* einer Seite. Dieser wird im Tab bzw. der Kopfzeile des Browsers angezeigt und darf nicht weggelassen werden. HTML-Dokumente ohne Titel sind nicht valide, d. h. der HTML-Validator¹ zeigt einen Fehler an. Die Angabe des Titels ist aber ohnehin immer im Sinne der Autor*innen, denn er wird von Suchmaschinen und in Browser-Lesezeichen (Bookmarks) verwendet. Der Titel sollte daher auch aussagekräftig gewählt werden.

```
1 <title>John Doe's Blog | Gedanken eines Namenlosen</title>
```

2.1.2 Basis-URL

Das `<base>`-Element in Kombination mit dem `href`-Attribut erlaubt es, den *Basis-URL* für eine Seite zu setzen. Somit wird allen relativen Verlinkungen (siehe dazu Abschnitt 2.4.1) dieser URL vorangestellt. Durch die Definition von

```
1 <base href="https://www.johndoe.com/">
```

¹<https://validator.w3.org/>

wird dieser URL jeder relativ angegebenen Quelle vorangestellt. Somit wird ein Bild, das durch `` (Details zu Bildern siehe Abschnitt 2.5.1) angegeben wurde, vom Webbrowser mit dem vollen URL, also `https://www.johndoe.com/image.png` eingebunden. Dies legt jedoch fest, dass die Seite unter der angegebenen Domain verfügbar ist, ein lokales Testen ist nicht mehr möglich.

Praktischer ist oft eine relative Angabe als Basis-URL. Liegen etwa alle Bilder im Verzeichnis `images`, so kann `/images/` als alleiniger Wert des `href`-Attributs verwendet werden, somit wird das Unterverzeichnis automatisch allen Bildangaben vorangestellt. Vorsicht jedoch, dies betrifft dann auch etwa Hyperlinks in der Seite, was nicht immer gewünscht sein mag.

2.1.3 Externe Verlinkungen und Style-Angaben

Über das Element `<link>` können *externe Dokumente* eingebunden werden. Die Syntax dazu wird in Vorlesung 3 beim Einbinden von Stylesheets erläutert.

Ebenso ist es möglich, *CSS-Formatierungen* im Kopfbereich über das `<style>`-Tag anzugeben. Dies wird ebenfalls in Vorlesung 3 thematisiert.

2.1.4 Meta-Angaben

Meta-Angaben erlauben es Autor*innen, zusätzliche Informationen über ein HTML-Dokument bereitzustellen. Sie lassen sich in drei Kategorien unterteilen: *Dokumentdaten*, *Angaben für den Webserver* und die verwendete *Zeichenkodierung*. Alle drei werden durch das `<meta>`-Element spezifiziert, jedoch jeweils durch ein anderes Attribut repräsentiert.

Dokumentdaten

Dokumentdaten sind Angaben, die den Dokumentinhalt genauer spezifizieren. Dies sind etwa Beschreibung, der Autor*innenname, Schlüsselwörter oder auch die verwendete Software (wenn die Seite automatisch erzeugt, d. h. nicht von Hand geschrieben wurde). Diese Kategorien werden beim Element `<meta>` mit dem `name`-Attribut angegeben. Der dazugehörige Wert steht in einem weiteren Attribut namens `content`. Mehrere Meta-Angaben für Dokumentendaten sehen wie folgt aus:

```
1 <meta name="description" content="Der Blog von John Doe">
2 <meta name="author" content="John Doe">
3 <meta name="keywords" content="Blog,John,Doe,News,Neuigkeiten">
4 <meta name="generator" content="JetBrains PhpStorm 2025.2.4">
```

Dokumentendaten haben heutzutage nur noch geringe Bedeutung. Früher wurden die Schlüsselwörter (`name="keywords"`) von Suchmaschinen zur Indizierung verwendet, der häufige Missbrauch hat jedoch dazu geführt, dass diese nicht mehr beachtet werden. Die Beschreibung (`name="description"`) und einige andere Metainformationen werden jedoch nach wie vor berücksichtigt [6].

Angaben für den Webserver

Wird eine Webseite vom Webserver zurückgegeben (Webserver Response), werden bestimmte, so genannte Header-Informationen mitgesendet. Sie enthalten z. B. die ver-

wendete Zeichenkodierung oder andere ähnliche Angaben. Diese Werte können in der HTML-Seite mittels *Webserver Angaben*, auch Pragma Direktiven genannt, überschrieben werden. Beispiele sind neben der verwendeten Zeichenkodierung ein Standard-Stylesheet oder die Weiterleitung auf einen neuen URL nach einer gewissen Zeit.

Um derartige Angaben zu machen, wird das `<meta>`-Element in Kombination mit dem `http-equiv`-Attribut verwendet, das die Art der Anweisung enthält. Wiederum wird ein zweites Attribut `content` verwendet, in dem die Werte für die Anweisungen enthalten sind. Die soeben genannten Anweisungen werden wie folgt umgesetzt:

```
1 <meta http-equiv="content-type" content="text/html; charset=utf-8">
2 <meta http-equiv="default-style" content="styles.css">
3 <meta http-equiv="refresh" content="20; URL=page2.html">
```

Zeile 1 teilt dem Browser mit, dass es sich um ein HTML-Dokument mit der UTF-8 Zeichenkodierung handelt. Zeile 2 gibt an, dass das bevorzugte Stylesheet die Datei `styles.css` ist (diese Angabe wird selten verwendet). Die Angaben in Zeile 3 bewirken, dass nach 20 Sekunden auf die Seite `page2.html` weitergeleitet wird.

Zeichenkodierung

Die *Angabe der Zeichenkodierung* ist essentiell, damit der Webbrowser weiß, wie er die im Dokument enthaltenen Zeichen anzuzeigen hat. Während es beim normalen Alphabet selten zu Problemen kommt, treten diese vor allem bei Sonderzeichen wie Umlauten auf. Die Angabe der korrekten Zeichenkodierung mit der soeben gezeigten Webserver Angabe verhindert dies, ist jedoch umständlich und schwer zu merken. Daher wurde mit HTML5 das Attribut `charset` (kurz für das englische „character set“) eingeführt, welches diese Angabe drastisch vereinfacht. Um die Zeichenkodierung nun anzugeben, kann (und sollte) folgendes Markup verwendet werden:

```
1 <meta charset="utf-8">
```

Zeichensätze und -kodierungen

Als *Zeichensatz* wird die Menge an Zeichen zur Darstellung von Schrift, Zahlen und Sonderzeichen am Computer bezeichnet. Im westlichen Raum dienen das lateinische Alphabet mit Umlauten und Sonderzeichen oder *Unicode* als Zeichensätze. Aufbauend auf diesen gibt es die verschiedensten *Zeichenkodierungen*. Sie geben an, durch wie viele Bytes ein Zeichen repräsentiert wird und welchen Wert es genau hat. Die im Webbereich wichtigste Zeichenkodierung ist UTF-8. Erwähnenswert sind aus historischen Gründen noch ASCII und ISO 8859.

Bei *ASCII* handelt es sich um einen 1963 eingeführten und zuletzt 1986 aktualisierten Standard [2] mit dem 95 Zeichen darstellbar sind (128 gibt es insgesamt, man spricht daher von einer 7-Bit Zeichenkodierung, da $2^7 = 128$). Enthalten sind das lateinische Alphabet in Groß- und Kleinschreibung, die zehn arabischen Ziffern sowie die wichtigsten Satzzeichen.

Aufbauend auf ASCII ist die Zeichenkodierung *ISO 8859-1* [5]. Es handelt sich um eine 8-Bit (= 1 Byte) Kodierung, somit ist Platz für 256 Zeichen ($2^8 = 256$). Die ersten 128 Zeichen entsprechen genau ASCII, die zweiten 128 sind mit deutschen Umlauten und diversen Sonderzeichen befüllt.

Neben ISO 8859-1 gibt es noch ISO 8859-2 bis ISO 8859-16. Diese Zeichenkodierungen unterscheiden sich jeweils durch die verwendeten Sonderzeichen. ISO 8859-15 entspricht ISO 8859-1, enthält aber bereits das Euro-Zeichen (€), ISO 8859-7 enthält griechische, ISO 8859-5 kyrillische Zeichen.

Schließlich gibt es mit *UTF-8* [13] die modernste Zeichencodierung. Sie basiert auf dem Unicode-Zeichensatz, der in Version 17.0 knapp 160.000 Zeichen der allermeisten bekannten Schriftsysteme enthält. UTF-8 bildet dies als 8-Bit Kodierung mit variabler Länge ab. Maximal möglich sind 32 Bit, also 4 Byte, die jedoch bei weitem nicht ausgenutzt werden. Wiederum sind die ersten 127 Zeichen mit ASCII identisch und daher abwärtskompatibel. Die deutschen Sonderzeichen haben jedoch andere Byte-Codes und stimmen nicht mit ISO 8859-1 überein. Somit ergeben sich falsche Darstellungen falls ein in ISO 8859-1 verfasstes Dokument vom Browser als UTF-8 dargestellt wird und umgekehrt.

Um Probleme zu vermeiden, sollte der Zeichensatz am Beginn angegeben werden und das Dokument selbst (also die Datei) im entsprechenden Format abgespeichert werden. Moderne Entwicklungsumgebungen und Editoren erlauben es, die Zeichenkodierung festzulegen.

2.2 Elemente zur Seitenstrukturierung

Die ab hier folgenden Elemente gehören alle zum Inhalt eines HTML-Dokuments, finden sich also innerhalb des `<body>`-Tags wieder.

Strukturelemente dienen zur Strukturierung eines HTML-Dokuments. Sie haben alle bestimmte Bedeutungen, mit denen einzelne Teile des Dokuments gekennzeichnet werden können. Ihnen allen gemeinsam ist die Tatsache, dass sie keine visuelle Änderung des Dargestellten bewirken, sondern lediglich zur logischen Gliederung dienen.

2.2.1 Inhaltsbereich

Das `<body>`-Element ist das oberste Gliederungselement. Es repräsentiert den Inhaltsbereich – alles was auf der Seite sichtbar angezeigt werden soll, muss sich innerhalb dieses Elements befinden.

2.2.2 Kopfbereich

Fast alle Websites haben einen *Kopfbereich* (engl. „header“). In der Regel befindet sich dort die Überschrift und/oder das Logo, aber auch eine Navigation kann zum Kopfbereich der Seite gehören. Um diesen Bereich zu kennzeichnen, wurde das `<header>`-Element (Abschluss mit `</header>`) eingeführt. Mit diesem Tag sollen alle Elemente einer Seite umschlossen werden, die von der Struktur her zum Kopfbereich gehören. Im einfachsten Fall ist das eine Überschrift mit zusätzlicher Tagline, wie folgendes Beispiel zeigt. Visuell ändert sich standardmäßig nichts, wie Abbildung 2.1 zeigt. Als Unterüberschrift (Tagline) sollte übrigens keine `<h1>`–`<h6>` Überschriften verwendet werden, da dies eine neue (ungewollte) Gliederungsebene erzeugt, sondern ein Absatz (`<p>`). Dieser sollte zusammen mit der Überschrift mit dem Element `<hgroup>` (siehe Abschnitt 2.2.9) gruppiert werden.

```
1 <header>
2   <hgroup>
3     <h1>John Doe's Blog</h1>
4     <p>Gedanken eines Namenlosen</p>
5   </hgroup>
6 </header>
```

Das `<header>`-Element kann aber auch in einem Teilabschnitt einer Webseite (begrenzt durch z. B. `<section>` oder `<article>`) die einleitenden Inhalte wie Abschnittsüberschriften, Inhaltsverzeichnisse oder Logos beinhalten.

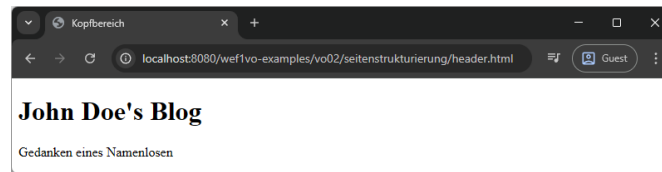


Abbildung 2.1: Überschrift und Unterüberschrift in einem `<header>`-Element verpackt. Der Browser stellt den Kopfbereich nicht explizit dar. Der Tag dient nur zur strukturellen Auszeichnung.

2.2.3 Fußbereich

Der *Fußbereich*, engl. „footer“, ist der Gegenpart zum Kopfbereich und enthält meist Informationen zu Autor*in, weiterführende Links oder Copyright-Informationen. Meist findet sich der Footer ganz am Ende eines Dokuments, jeder Abschnitt kann aber genauso seinen eigenen Fußbereich haben, wenn es nötig ist. Das HTML-Element hierfür lautet `<footer>` bzw. `</footer>`.

Das Beispiel demonstriert einen kurzen Fußbereich am Ende eines stark vereinfachten Dokuments:

```
1 <body>
2   <h1>John Doe's Blog</h1>
3   ...
4   <footer>
5     <p>&copy; 2025 by John Doe</p>
6   </footer>
7 </body>
```

2.2.4 Navigation

Auch für *Navigationen* ist in der HTML-Definition ein Element vorgesehen: `<nav>` (abgeschlossen durch `</nav>`). Es soll dazu verwendet werden, um die Hauptnavigation(en) auf der Seite zu kennzeichnen. Nicht jeder einzelne Link soll und muss in einem `<nav>`-Block stehen.

Navigationen werden üblicherweise mit Aufzählungslisten (siehe Abschnitt 2.3.6) realisiert und später entsprechend visuell angepasst. Das Markup für eine Navigation kann etwa so aussehen (mehr zu den verwendeten Hyperlinks in Abschnitt 2.4.1).

```
1 <nav>
2   <ul>
3     <li><a href="index.html">Home</a></li>
4     <li><a href="about.html">About</a></li>
5     <li><a href="projects.html">Projects</a></li>
6   </ul>
7 </nav>
```

2.2.5 Abschnitte

Eine Webpage besteht selten aus einem langen Themenblock, meist sind es mehrere thematisch ähnliche Dinge, die auf einer Seite zu finden sind. Um diesem Umstand

Rechnung zu tragen, wurde mit HTML5 das `<section>`-Element (Abschluss durch `</section>`) eingeführt. Es kennzeichnet einen inhaltlich zusammengehörigen Bereich innerhalb einer Seite. Typischerweise enthält ein *Abschnitt* eine Überschrift und dazugehörigen Text:

```
1 <section>
2   <h2>Aktuelles</h1>
3   <p>In einem vor kurzem veröffentlichten Bericht&nbsp;...</p>
4 </section>
5 <section>
6   <h2>Projekte</h1>
7   <p>Es folgt eine Liste der aktuellsten Projekte.</p>
8 </section>
```

2.2.6 Artikel

Artikel funktionieren ähnlich wie Abschnitte, sollen jedoch als eigenständiger logischer Teil funktionieren (könnten also losgelöst ohne den umgebenden Inhalt stehen). Typischerweise finden sich Artikel auf Blogs oder Websites von Zeitungen in denen sie, wie der Name schon vermuten lässt, einzelne Artikel repräsentieren. Ein Artikel wird durch `<article>` und `</article>` gekennzeichnet.

```
1 <article>
2   <header>
3     <hgroup>
4       <h2>Die Hagenberg Story</h2>
5       <p>Alles über den Studienort im Mühlviertel</p>
6     </hgroup>
7   </header>
8   <p>Es begann alles im Jahr 1993&nbsp;...</p>
9 </article>
```

2.2.7 Nebenabschnitte

In Zeitungen existieren bei Artikeln nebenbei oft Boxen mit zusätzlichem Inhalt, der nur lose im Zusammenhang mit dem eigentlichen Text steht. Dies kann z. B. bei einem Artikel über Hagenberg eine Box mit demografischen Daten über die Gemeinde sein.

Für diese Zwecke wurde in HTML5 der Tag `<aside>` (Abschluss mit `</aside>`) eingeführt. Er kann des Weiteren für Zitate aus dem Text, Werbung oder auch kleine Navigationsboxen (mit darin enthaltenem `<nav>`-Element) verwendet werden. `<aside>` dient auch für Sidebars in Layouts (mehr dazu in Vorlesung 5) oder zur Anzeige von Werbung.

Ein Beispiel für eine korrekte Verwendung, aufbauend auf dem vorigen Markup:

```
1 <article>
2   <header>
3     <hgroup>
4       <h2>Die Hagenberg Story</h2>
5       <p>Alles über den Studienort im Mühlviertel</p>
6     </hgroup>
7   </header>
8   <p>Es begann alles im Jahr 1993&nbsp;...</p>
9   <aside>
```

```

10    <p>Die Gemeinde Hagenberg hat 3.163 Einwohner*innen&nbsp;...</p>
11    </aside>
12 </article>

```

2.2.8 Überschriften

Wie bereits in Vorlesung 1 kurz erwähnt, gibt es sechs Ebenen von Überschriften, die durch die Elemente `<h1>`, `<h2>`, `<h3>`, `<h4>`, `<h5>` und `<h6>` (mit den jeweils abschließenden Elementen) repräsentiert werden. Diese Überschriften erzeugen inhaltliche Strukturen (die so genannte Dokumenten-Outline), wie sie auch aus Textdokumente (z. B. dieses Skriptum) bekannt und ersichtlich sind. Folgende Abfolge von Überschriften

```

1 <h1>Überschrift 1</h1>
2 <h2>Überschrift 1.1</h2>
3 <h3>Überschrift 1.1.1</h3>
4 <h2>Überschrift 1.2</h2>
5 <h2>Überschrift 1.3</h2>
6 <h1>Überschrift 2</h1>

```

erzeugt die folgende Struktur:

1. Überschrift 1
 - 1.1. Überschrift 1.1
 - 1.1.1. Überschrift 1.1.1
 - 1.2. Überschrift 1.2
 - 1.3. Überschrift 1.3
2. Überschrift 2

Strukturelemente wie `<section>`, `<article>` und dergleichen, haben keinen Einfluss auf die Outline. Die Überschriften werden in aufsteigender Reihenfolge weiterverwendet.

```

1 <h1>John Doe's Blog</h1>
2 <section>
3   <h2>Artikel zu Katzen</h2>
4   <article>
5     <h3>Eine Katze namens Nobody</h3>
6     <p>Es war einmal&nbsp;...</p>
7   </article>
8 </section>

```

Es ergibt sich dadurch folgende (erwartete) Gliederung:

1. John Doe's Blog
 - 1.1. Artikel zu Katzen
 - 1.1.1. Eine Katze namens Nobody

Aus Gründen der Accessibility sollte jede Seite mindestens eine `<h1>`-Überschrift besitzen. Es sollten in der Hierarchie keine Überschriftenebenen übersprungen werden, d. h. auf `<h1>` sollte nie direkt `<h3>` folgen.

2.2.9 Überschriftengruppen

Soll eine Überschrift um eine Unterüberschrift oder Tagline ergänzt werden, so ist es wichtig, für dieses Element *keinen* weiteren Überschrift-Tag zu verwenden. Dies würde automatisch eine Ebene in der Outline erzeugen und dies ist nicht gewünscht. Vielmehr soll ein einfacher Absatz (<p>) zum Einsatz kommen. Damit dieser jedoch semantisch zur Überschrift zuordenbar ist, werden die Überschrift und die Tagline(s) mit einem <hgroup>-Element gruppiert.

Dies war etwa bei der Überschrift des Artikels in Abschnitt 2.2.6 der Fall.

```
1 <article>
2   <h1>John Doe's Blog</h1>
3   <header>
4     <hgroup>
5       <h2>Die Hagenberg Story</h2>
6       <p>Alles über den Studienort im Mühlviertel</p>
7     </hgroup>
8   </header>
9   <p>Es begann alles im Jahr 1993&nbsp;...</p>
10 </article>
```

2.3 Elemente zur Textstrukturierung

Während die Elemente in Abschnitt 2.2 zur Gliederung der gesamten HTML-Seite verwendet wurden, kommen die folgenden Elemente zur Strukturierung von Textinhalten zum Einsatz.

2.3.1 Absätze

Das wohl am weitesten verbreitete Element zur Textstrukturierung ist der Textabsatz. Er wird mit <p> bzw. </p> erzeugt und kann neben Text wiederum eine Vielzahl anderer Elemente enthalten. Ein einfacher Absatz sieht wie folgt aus:

```
1 <p>Etwas Text in einem Absatz.</p>
```

2.3.2 Adressen

Das <address>-Element (Vorsicht, englische Schreibweise mit zwei „d“, Abschluss mittels </address>) wird dazu verwendet, *Kontaktinformationen* auf einer Seite anzugeben. Diese Kontaktinformationen können weiterführende Links, E-Mail- oder in Einzelfällen auch Postadressen sein. Es wird in [9, Abschnitt 4.3.10] ausdrücklich darauf hingewiesen, das Element ausschließlich für Kontaktadressen zu verwenden. Andere Adressen (z. B. Postadressen für eine Wegbeschreibung) sollen regulär in einen Absatz (<p>) gestellt werden. Ein Beispiel für die korrekte Verwendung mit einem Hyperlink auf eine E-Mail-Adresse als Kontaktmöglichkeit (siehe Abschnitt 2.4.1):

```
1 <address>
2   This blog is run by <a href="mailto:john.doe@johndoe.com">John Doe</a>
3 </address>
```

Eine Postadresse wäre nur dann im <address>-Element erlaubt, wenn sie die (einzig) relevante Kontaktmöglichkeit darstellt.

2.3.3 Hauptinhalt

Das `<main>`-Element (abgeschlossen durch `</main>`) kennzeichnet den *Hauptinhalt* in einem Dokument, also den Teil, in dem die wichtigsten Inhalte vorkommen. Im Gegensatz zu `<header>`, `<footer>`, `<nav>` und `<aside>` darf es nur ein einziges, sichtbares `<main>`-Element auf der Seite geben, dieses darf auch nicht innerhalb der soeben genannten Elemente vorkommen, sondern ist in der Regel direkt innerhalb des `<body>`-Elements platziert. Das Element hat auch keinen Einfluss auf die Dokumenten-Outline.

Oft gestaltet sich die Gliederung zusammen mit den anderen Strukturelementen wie folgt:

```
1 <header>...</header>
2 <nav>...</nav>
3 <main>
4   <h1>Die Geschichte von Hagenberg</h1>
5   <p>Es begann alles im Jahr 1993&nbsp;&nbsp;&nbsp;&...</p>
6 </main>
7 <footer>...</footer>
```

2.3.4 Suchbereich

Mit dem Element `<search>` (abgeschlossen durch `</search>`) wird der Bereich einer Seite gekennzeichnet, in dem eine Suche oder eine Filterung durchgeführt werden kann. Meist enthält er ein Suchfeld vom Typ `search` (mehr zu Formularen in Abschnitt 2.6). Die Suchergebnisse selbst sollten *nicht* in diesem Bereich sondern im Hauptbereich der Seite (z. B. in einem `<output>`-Element) dargestellt werden. Das Element ändert nichts am Aussehen der Seite, aber hilft Tools zur Barrierefreiheit, Suchbereiche eindeutig zu identifizieren.

Ein Suchfeld in einem Suchbereich kann wie folgt aussehen:

```
1 <search>
2   <form action="..." method="get">
3     <label for="suche">Suche:</label>
4     <input type="search" id="suche">
5     <button type="submit">Suchen</button>
6   </form>
7 </search>
```

2.3.5 Allgemeiner Bereich

Die allgemeinste Form eines Bereich-Elements ist das `<div>`-Element. Außer eine neue Zeile zu beginnen, hat es keine spezifische Bedeutung. Eine solche erlangt es in der Regel erst durch eine Formatierung mittels CSS. Dieses Element kann wiederum viele andere Elemente beinhalten und sollte immer dann verwendet werden, wenn sonstige gruppierende Elemente nicht passen bzw. auch die Dokumenten-Outline nicht beeinflusst werden soll. Das folgende Markup zeigt ein `<div>`-Element mit einer CSS-Klassenangabe und etwas Inhalt. Mehr zu CSS in Vorlesung 3.

```
1 <div class="catchphrase">
2   <p>John Doe: you won't even know I'm here</p>
3 </div>
```

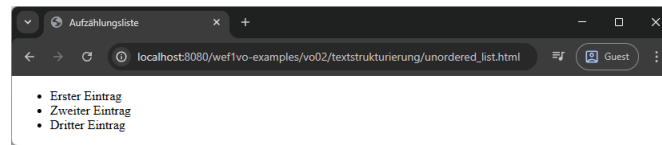


Abbildung 2.2: Eine Aufzählungsliste mit drei Einträgen.

2.3.6 Listen

Listen dienen zur Gruppierung bzw. Aufzählung von Text. Es existieren drei unterschiedliche Arten.

Aufzählungslisten

Aufzählungslisten, im Englischen „unordered lists“ oder „bullet lists“ dienen zur strukturierten Darstellung mehrerer Punkte. Jeder Eintrag wird mit einem Aufzählungszeichen (Bullet) versehen. Eine Aufzählungsliste wird mit dem `<ul>`-Element geöffnet und mit `</ul>` beendet.

Jeder einzelne Eintrag in der Aufzählungsliste (engl. „list item“) wird zwischen `<li>` und `</li>` gestellt.

Das Markup für eine Aufzählungsliste sieht somit wie folgt aus, das Ergebnis ist in Abbildung 2.2 dargestellt.

```
1 <ul>
2   <li>Erster Eintrag</li>
3   <li>Zweiter Eintrag</li>
4   <li>Dritter Eintrag</li>
5 </ul>
```

Eine semantische Alternative zu `<ul>` ist `<menu>`, welches verwendet werden kann, wenn etwa eine Toolbar mit klickbaren Buttons realisiert wird.

Nummerierte Listen

Nummerierte Listen (engl. „ordered lists“) funktionieren ähnlich wie Aufzählungslisten. Anstatt der Bullet Points sind die Einträge nummeriert, es wird das `<ol>`-Element als Beginn der Liste und das `</ol>` zum Abschluss verwendet. Einträge werden wiederum mit dem `<li>`-Element gekennzeichnet. Über die Attribute **reversed** und **start** können die Reihenfolge umgedreht bzw. der Startwert festgelegt werden.

Das Markup für eine nummerierte Liste und das Ergebnis in Abbildung 2.3:

```
1 <ol>
2   <li>Erster Eintrag</li>
3   <li>Zweiter Eintrag</li>
4   <li>Dritter Eintrag</li>
5 </ol>
```

Definitionslisten

Definitions- oder Beschreibungslisten dienen zur Beschreibung von Begriffen. Die Liste wird mit `<dl>` begonnen und durch `</dl>` abgeschlossen. Innerhalb der Liste gibt es

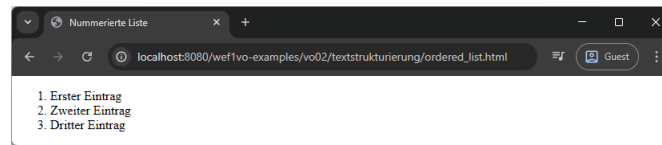


Abbildung 2.3: Eine nummerierte Liste mit drei Einträgen.

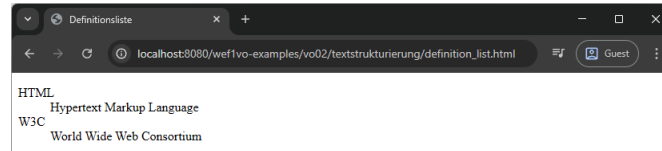


Abbildung 2.4: Eine Definitionsliste mit zwei Einträgen.

zwei Tags, die verwendet werden müssen. Der zu beschreibende Begriff (engl. „definition term“) wird mit `<dt>` `</dt>` ausgezeichnet, die Beschreibung dazu (engl. „definition description“) durch `<dd>` `</dd>`. Die Reihenfolge (`<dt>` vor `<dd>`) muss eingehalten werden.

Das Markup für eine Definitionsliste und das Ergebnis in Abbildung 2.4:

```
1 <dl>
2   <dt>HTML</dt>
3   <dd>Hypertext Markup Language</dd>
4   <dt>W3C</dt>
5   <dd>World Wide Web Consortium</dd>
6 </dl>
```

2.3.7 Vorformatierter Text

Mit dem Element `<pre>` (Abschluss mit `</pre>`) wird ein Block aus *vorformatiertem Text* angegeben. Der Text wird mit einer Festbreitenschrift dargestellt, Struktur wird durch typografische Konventionen anstatt durch Elemente definiert. So werden Leerzeichen und Tabulatoren nicht entfernt und Zeilenumbrüche durch tatsächliche Zeilenumbrüche innerhalb des Blocks erzielt.

Abbildung 2.5 stellt das folgende Beispielmarkup dar:

```
1 <pre>
2   Dieser Text   erscheint
3 wirklich
4   so   seltsam.
5 </pre>
```

Vorsicht: Wird der `<pre>`-Block etwa aus Gründen der Lesbarkeit im HTML-Quellcode mittels Tabulatoren oder Leerzeichen eingerückt, so wird dies auch bei der Ausgabe sichtbar. `<pre>` sollte daher immer uneingerückt am Anfang der Zeile stehen.

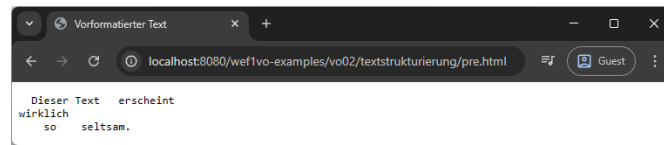


Abbildung 2.5: Vorformatierter Text. Leerräume und Zeilenumbrüche bleiben genauso erhalten, wie sie im Markup eingegeben wurden.

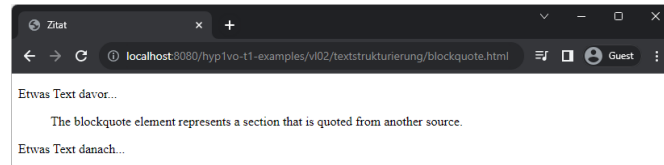


Abbildung 2.6: Das Zitat wird vom Browser standardmäßig eingerückt.

2.3.8 Horizontale Linie

Um einen thematischen Wechsel innerhalb eines Abschnittes (<section>) zu kennzeichnen, kann eine *horizontale Trennlinie* verwendet werden. Sie ist ein Standalone-Element und wird mit <hr> erzeugt. Oft wird sie zwischen zwei aufeinanderfolgenden Absätzen (<p>) platziert, es sind aber auch andere Strukturelemente wie Listen oder Bereiche (<div>) möglich. Zwischen <section>-Elementen wird in der Regel keine horizontale Linie platziert, da diese ohnehin einen thematischen Wechsel kennzeichnen.

```
1 <section>
2   <h1>Über Hagenberg</h1>
3   <p>Die Gemeinde Hagenberg hat nach wie vor eine starke ländliche Ausprägung.</p>
4   <hr>
5   <p>Die 1993 eröffnete Fachhochschule repräsentiert den modernen Teil der Gemeinde.
6   </p>
7 </section>
```

2.3.9 Zitate

Um Text aus verschiedenen Quellen, wie etwa Büchern oder Websites zu zitieren, existiert das `<blockquote>`-Element (Abschluss `</blockquote>`). Durch die Angabe des `cite`-Attributes beim Element kann die Quelle angegeben werden. Im folgenden Beispiel wird die Beschreibung des Elements aus [9, Abschnitt 4.4.4] zitiert, das Ergebnis ist in Abbildung 2.6 ersichtlich.

```
1 <p>Etwas Text davor&nbsp;&nbsp;&...</p>  
2 <blockquote cite="https://html.spec.whatwg.org/multipage/grouping-content.html#the-  
   blockquote-element">  
3   <p>The blockquote element represents a section that is quoted from another source.  
   </p>  
4 </blockquote>  
5 <p>Etwas Text danach&nbsp;&nbsp;&...</p>
```

2.4 Links und Elemente auf Textebene

Während Elemente zur Textstrukturierung einem Text eine bestimmte Bedeutung geben, arbeiten Elemente auf der Textebene direkt auf bzw. in dem Text und erfüllen dort diverse Aufgaben. Sehr wichtig sind dabei Hyperlinks, auf der Textebene arbeiten jedoch viele unterschiedliche Elemente.

2.4.1 Verweise

Verweise oder *Hyperlinks* sind das Kernstück von Hypertext – die Möglichkeit zu anderen Dokumenten zu verlinken. Zuständig in HTML ist hierfür das `<a>`-Element, welches für Anker, im Englischen „anchor“ steht. Um das Verweisziel zu spezifizieren ist immer das `href`-Attribut (für „hypertext reference“) notwendig. Das Ziel kann dabei auf verschiedene Arten angegeben werden.

Relative Links

Relative Links gehen immer vom Speicherort der HTML-Datei aus, in welcher der Link platziert ist. Bei relativen Links sind nur die Zieldatei und ggf. darüber oder darunter liegende Verzeichnisse angegeben. Innerhalb von Websites sollte prinzipiell relativ verlinkt werden, da relative Links unabhängig vom genauen Speicherort immer funktionieren.

Der folgende Link verweist auf eine Datei, die im gleichen Verzeichnis wie die aktuelle Datei liegt.

```
1 <a href="datei1.html">Ein Link</a>
```

Die folgenden zwei Beispiele verweisen auf Dateien, die einmal im Unterverzeichnis `docs` bzw. zwei Verzeichnisebenen höher liegen.

```
1 <a href="docs/datei2.html">Ein Link</a>
2 <a href="../../datei3.html">Ein Link</a>
```

Absolute Links

Absolute Links verweisen in der Regel auf eine externe Ressource. Im Gegensatz zu relativen Links sind hier das Protokoll (z.B. `http` oder `https`) und die volle Domain-Angabe vorhanden. Auch wenn auf Unterseiten der eigenen Website absolut verlinkt werden kann, empfiehlt sich dies nicht. Sollte z.B. die Domain gewechselt werden, so müssen alle Links korrigiert werden.

Die folgenden Beispiele zeigen externe HTTP- (bzw. HTTPS-) Links und einen unter Verwendung von FTP (File Transfer Protocol). Anmerkung: moderne Browser haben die FTP-Unterstützung aus Sicherheitsgründen entfernt.

```
1 <a href="http://speedtest.tele2.net/">HTTP-Link</a>
2 <a href="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/a">
  HTTPS-Link</a>
3 <a href="ftp://ftp.cert.org/">FTP-Link</a>
```

Relative Links mit führendem Slash: root-relative Links

Wird einem relativen Link ein Schrägstrich vorangestellt, wird dieser zu einem root-relativen Link, der sich auf die unterste (Root-) Ebene der Verzeichnisstruktur bezieht. Angenommen in einer Datei `https://johndoe.com/blog/index.html` befinden sich die folgenden Links:

- `<a href="media/projekte.html">Projekte</a>` verweist auf den URL `https://johndoe.com/blog/media/projekte.html`, denn es ist ein relativer Link, der ausgehend vom Verzeichnis `blog` noch eine Ebene tiefer ins Verzeichnis `media` geht und dort auf `projekte.html` verweist.
- `<a href="/media/projekte.html">Projekte</a>` verweist hingegen auf `https://johndoe.com/media/projekte.html`, denn der führende Slash geht aus vom Root der Seite (`https://johndoe.com/`) aus und von dort weiter ins Unterverzeichnis `media` mit der darin enthaltenen Datei `projekte.html`.

E-Mail-Adressen

Um einen Link auf eine *E-Mail-Adresse* zu setzen, wird diese unter Voranstellung des `mailto:-`Pseudoprotokolls in das `href`-Attribut eines Ankers gesetzt. Ein einfacher E-Mail-Link kann daher folgendermaßen erzeugt werden:

```
1 <a href="mailto:john.doe@johndoe.com">Send an e-mail!</a>
```

Ein Klick auf den Link öffnet in der Regel das im Betriebssystem definierte E-Mail-Programm und trägt die Adresse in das „An:“ Feld ein.

Zusätzliche Angaben bei E-Mail-Links

Zusätzlich können auch noch weitere Felder vorausgefüllt werden. Um weitere Daten mitzugeben, muss hinter die Adresse ein `?` gestellt werden. Dies zeigt an, dass Parameter im Format `name=wert` folgen. Auch mehrere solcher Parameter sind möglich, diese müssen mit `&` voneinander getrennt werden. Da dieses Zeichen in HTML aber reserviert ist, muss die entsprechende Entität, also `&`; stattdessen verwendet werden.

Folgende Parameter können mit dieser Syntax mitgegeben werden:

- **to:** Der*die primäre Adressat*in. Diesen Parameter anzugeben ist nicht nötig, er wird ohnehin mit der E-Mail-Adresse befüllt.
- **cc:** Ein*e Adressat*in im CC-Feld.
- **bcc:** Ein*e Adressat*in im BCC-Feld (wird von Empfänger*innen nicht gesehen).
- **subject:** Die Betreffzeile des E-Mails.
- **body:** Der Textbereich (Inhalt) des E-Mails.

Leerzeichen müssen ersetzt („escaped“) werden. Dies geschieht durch die Verwendung von `%20`.

Mit folgendem Markup wird ein E-Mail mit vorgegebenem Text und Betreff an eine Person und an eine weitere im CC geschickt:

```
1 <a href="mailto:john.doe@johndoe.com?cc=survey@johndoe.com&amp;subject=Feedback%20to%20your%20site&amp;body=Dear%20John...">Send Feedback!</a>
```

Lokale Dateien

Es ist auch möglich, direkt auf *Dateien im Dateisystem* zu verlinken. Dazu wird der Pfad im Dateisystem angegeben, vorangestellt wird das `file:///` Protokoll. Die einzelnen Ordnerhierarchien sind mit Schrägstrichen (Slashes), nicht Backslashes zu trennen.

Ein Beispiel für eine Verlinkung ins Dateisystem:

```
1 <a href="file:///C:/Dokumente/HTML/index.html">Ein File-Link</a>
```

Vorsicht: Derartige Links funktionieren in der Regel nur lokal auf ein und demselben Rechner. Wird die HTML-Datei, in der der Link enthalten ist, auf einem Webserver gehostet oder auf einem anderen Computer geöffnet, verweist sie auf einen Pfad, der so auf diesem Rechner wahrscheinlich nicht existieren wird. Daher sollten solche Links nur in Ausnahmefällen verwendet werden.

Interne Links

Auch innerhalb einer HTML-Datei kann mittels Hyperlinks navigiert werden. Zunächst muss an einer Stelle ein Anker definiert werden. Dies geschieht wiederum mit dem `<a>`-Element und mit Hilfe des `id`-Attributs. Dieses ist ein globales Attribut, das einem Element eine eindeutige Bezeichnung gibt. Der Wert des Attributs kann beliebig gewählt werden.

Um diesen Anker „anzuspringen“, wird wiederum das `<a>`-Element, diesmal in Kombination mit dem bekannten `href`-Attribut verwendet. Der Wert ist der Name des zuvor erstellten Ankers mit vorangestelltem `#`-Zeichen.

Ein Beispiel für einen *internen Link*, der auf einen Anker zeigt:

```
1 <h1><a id="anfang">John Doe's Blog</a></h1>
2 ...
3 <p><a href="#anfang">Zum Anfang springen</a></p>
```

Um einen internen Anker zu definieren, muss jedoch nicht zwingend ein `<a>`-Element mit einem `id`-Attribut zum Einsatz kommen. Vielmehr kann letzteres auch direkt beim anzuspringenden Element hinzugefügt werden. Das obige Beispiel kann (und sollte der Einfachheit halber) auch wie folgt realisiert werden:

```
1 <h1 id="anfang">John Doe's Blog</h1>
2 ...
3 <p><a href="#anfang">Zum Anfang springen</a></p>
```

2.4.2 Weitere Elemente auf Textebene

Zur Formatierung von Text gibt es eine Reihe von Elementen, von denen im Folgenden der Großteil kurz aufgelistet und erklärt werden. Eine vollständige Erklärung findet sich im HTML Living Standard².

: Dieses Element wird verwendet, um Textstellen zu betonen (engl. „emphasize“). Browser stellen dies standardmäßig kursiv dar.

: Mit diesem Element wird die hohe Wichtigkeit einer Textstelle gekennzeichnet. Im Normalfall wird diese durch Fettschrift dargestellt.

<small>: Wird verwendet, um Textstellen als „Kleingedrucktes“ zu markieren. Beispiele sind etwa Disclaimer oder Copyright-Richtlinien. Dieses Element sollte nicht dazu verwendet werden um Text kleiner zu schreiben (auch wenn es Browser standardmäßig kleiner rendern), es ist auch nicht das Gegenteil von ****.

²<https://html.spec.whatwg.org/multipage/text-level-semantics.html>

<s>: Dieses Element kennzeichnet Textpassagen, deren Inhalte nicht mehr zeitgemäß oder relevant sind. Browser rendern dies standardmäßig durchgestrichen.

<cite>: Kann verwendet werden, um Titel von Werken wie Büchern oder wissenschaftlichen Papers zu kennzeichnen. Es wird oft gemeinsam mit dem **<blockquote>**-Element verwendet.

<q>: Dieses Element kennzeichnet ein kurzes Zitat oder Teile eines Zitats im Textfluss. Wie auch bei **<blockquote>** kann das **cite**-Attribut zur Quellenangabe verwendet werden.

<dfn>: Dieses Element kennzeichnet einen Ausdruck, der näher beschrieben werden soll (z.B. eine Abkürzung). Die Erklärung dazu kann u.a. mit dem **title**-Attribut angegeben werden.

<abbr>: Mit diesem Element können Abkürzungen (engl. „abbreviations“) oder Akronyme erläutert werden. Die Erklärung des markierten Ausdrucks soll im **title**-Attribut festgehalten werden.

<data>: Dieses Element erlaubt es, zu Inhalten noch maschinenlesbare Informationen zu speichern, die für Benutzer*innen selbst nicht relevant sind. Diese werden mit dem **value**-Attribut angegeben. So etwa für einen Benutzer*innennamen und dazugehöriger ID: `<data value="jdoe">John Doe</data>`.

<time>: Wird verwendet, um Datum oder Uhrzeit anzugeben. Der Inhalt kann ein beliebiges, menschenlesbares Format sein, mit dem Attribut **datetime** kann eine maschinenlesbare Version hinterlegt werden.

<code>: Dieses Element kann verwendet werden, um Code-Fragmente auszuzeichnen. Oft wird es zusätzlich von einem **<pre>**-Element umschlossen.

<var>: Mit diesem Element können mathematische Variablen im Textfluss gekennzeichnet werden.

<samp>: Wird verwendet, um die Ausgabe (engl. „sample output“) von Computerprogrammen zu kennzeichnen.

<kdb>: Kennzeichnet Input von Benutzer*innen über das Keyboard oder andere Eingabegeräte.

<sub> und <sup>: Werden für Tief- und Hochstellung von Text verwendet. Dabei ist es wichtig, die Elemente nur dann zu verwenden, wenn Sub- oder Superscript Teil einer typographischen Konvention ist und dies dadurch zum Ausdruck gebracht werden muss (z. B. um „M^{me}“, die Kurzschreibweise für das französische Wort „Madame“ korrekt zu bezeichnen, oder bei mathematischen Formeln).

<i>: Wird benützt, um Text in einer anderen Stimmung oder anderen Sprache zu kennzeichnen. Bei letzterem sollte das **lang**-Attribut verwendet werden, um die Sprache anzugeben. Browser stellen dies für gewöhnlich kursiv dar.

: Hebt Text hervor, ohne ihn dabei extra zu betonen. Dies kann verwendet werden, um bestimmte Schlüsselwörter einfach hervorzuheben, ohne dabei ihre Bedeutung zu sehr zu steigern. Dieses Element wird nur selten benötigt, meist ist **** die bessere Alternative. Dieses Element wird von Browsern in der Regel fett gerendert.

<u>: Dieses Element soll dazu verwendet werden, um eine Anmerkung zu einem Textteil auszudrücken, die jedoch nicht ausdrücklich erwähnt wurde. Ein Beispiel wäre die Kennzeichnung eines falsch geschriebenen Wortes – dieses würde man auf Papier etwa unterstreichen. Auch Browser stellen das **<u>**-Element unterstrichen dar. Aufgrund der leichten Verwechslung mit Hyperlinks sollte es sparsam bis gar nicht eingesetzt werden.

<mark>: Dieses Element kann benützt werden, um Teile im Text hervorzuheben, d. h. zu markieren, um die Aufmerksamkeit der Leser*innen auf diese Stelle zu richten. Es handelt sich quasi um einen mit dem Textmarker gekennzeichneten Bereich – Browser stellen sie analog dazu gelb hinterlegt dar.

: Dieses Element ist ein allgemeines Markup-Element ohne spezielle Bedeutung. Es verhält sich ähnlich wie **<div>** wird aber im Fließtext verwendet und nicht als eigener Bereich.

2.5 Bilder

Bilder sind zentraler Bestandteil einer Webseite. Mit den modernen Mitteln der neuesten HTML-Versionen ist deren Handhabung im Browser sehr flexibel geworden.

2.5.1 Bilder einfügen

Um *Bilder* in ein HTML-Dokument einzubinden, wird das ****-Element verwendet. Es ist ein Standalone-Element und besitzt mehrere wichtige Attribute, von denen **src** und **alt** verpflichtend verwendet werden müssen. **src** spezifiziert den Speicherort des anzuzeigenden Bildes (Quelle), mit **alt** wird ein Alternativtext spezifiziert, der angezeigt wird, wenn das Bild nicht geladen werden kann bzw. ausgegeben wird, wenn Personen mit visueller Beeinträchtigung die Website von einem Screenreader vorgelesen bekommen.

Das einfachste Markup, um ein Bild einzufügen, sieht also folgendermaßen aus. Das Ergebnis ist in Abbildung 2.7 dargestellt.

```
1 
```

Es empfiehlt sich, beim Einbinden des Bildes auch dessen Größe mitanzugeben. Dies ist mithilfe der Attribute **width** und **height** möglich. Diese Attribute sind zwar nicht zwingend notwendig, denn der Browser kann auch aus den Bilddaten die Dimensionen auslesen. Bekommt er diese Information aber bereits vorab, kann er beim Aufbau/Laden

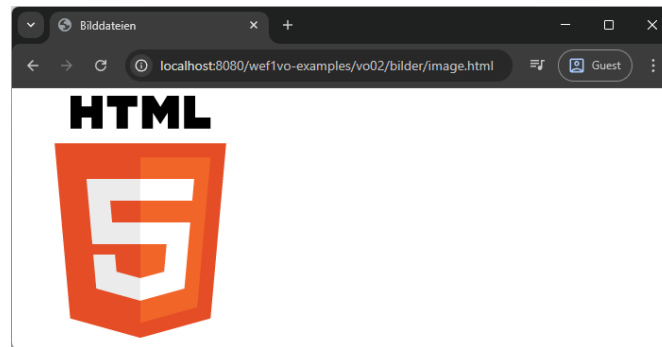


Abbildung 2.7: Die im `src`-Attribut angegebene Bilddatei `html5logo-small.png` wird angezeigt. Der Inhalt des `alt`-Attributs ist nicht sichtbar.

der Seite bereits den exakten Platz für das Bild reservieren. Die beiden Attribute sollten nicht zur Skalierung von Bildern verwendet werden sondern immer die tatsächliche Größe angeben.

Das Markup mit entsprechenden Längen- und Breitenangaben in Pixel (eine Angabe von Einheiten oder Prozent ist nicht möglich). Das Ergebnis sieht nach wie vor wie in Abbildung 2.7 aus.

```
1 
```

2.5.2 Pixeldichte und Auflösung berücksichtigen

Es besteht die Möglichkeit, im `<img>`-Element nicht nur ein einziges Quellbild anzugeben, sondern Alternativen, die dem Browser zur Auswahl zur Verfügung stehen – je nachdem auf welchem Display der Inhalt angezeigt wird oder wie groß der Anzeigebereich (Viewport) ist. Verantwortlich dafür ist das `srcset`-Attribut, das für diese Zwecke verschieden aussehen kann.

Angaben für Pixeldichte

Um etwa für hochauflösende Displays (wie das Retina-Display von Apple) entsprechend besser aufgelöste Grafiken zur Verfügung zu stellen, können im `srcset`-Attribut mehrere Grafikdateien durch Komma getrennt angegeben werden. Nach dem Namen der Dateien erfolgt die Angabe eines *Pixeldichte*-Werts. *1x* steht für reguläre Displays, *2x* für hochauflösende Displays. Diese Angaben können auch noch weiter erhöht werden. Die Angabe von `src` bleibt weiterhin und dient als Fallback, falls der Browser das neue Attribut noch nicht kennt.

Ein Bild in zwei Größen kann dann wie folgt eingebunden werden, der Browser wählt automatisch das passendere Bild:

```
1 
```

Angaben für Auflösungen

Die Angabe der Pixeldichte allein ist jedoch in vielen Fällen unflexibel. Oft möchte man je nach aktueller Größe des Browserfensters (oder auf mobilen Geräten des Viewports) das passende Bild anzeigen, d. h. mehrere Versionen in verschiedenen *Auflösungen* hinterlegen. Hierbei wird im **srcset**-Attribut wieder eine kommagetrennte Liste mit Grafikdateien angegeben, danach folgt aber jeweils die Breite des Bildes in der *w*-Syntax. Ist dieses z. B. 512px Breit, wird 512*w* geschrieben etc.

Für das folgende Bild liegen vier Größen vor, der Browser wählt die passende aus. Wie genau dies passiert, obliegt dem Browser. Google Chrome wählt etwa bei lokal geöffneten Seiten immer das größte Bild.

```
1 
```

Angabe der Größe je nach Auflösung

Die soeben gezeigte Methode hat jedoch den Effekt, dass das jeweils ausgewählte Bild (ohne weitere Angaben) auf die maximal mögliche Breite skaliert wird. Um eine tatsächliche *Größe* anzugeben, kann zusätzlich mit dem **sizes**-Attribut gearbeitet werden. Dieses kann, je nach Viewport-Breite verschiedene Größen setzen. Zunächst kann eine Angabe eine Breitenangabe in Klammern gesetzt werden, danach wird die Größe spezifiziert. Weitere, durch Komma getrennte Angaben sind möglich.

Im folgenden Beispiel wird bis zu eine Viewport-Breite von 600 Pixeln das Bild auf 100 % des Viewports skaliert (100vw – mehr dazu in Vorlesung 3), bis zu einer Breite von 1.200 Pixeln auf 75 % des Viewports und ab dieser Breite wird eine fixe Größe von 1024 Pixeln eingestellt.

```
1 
```

2.5.3 Art Direction mit <picture>

Während das **srcset**-Attribut beim ****-Element die Funktion hat, verschiedene Versionen desselben Bildes je nach Auflösung oder Display anzubieten, gibt es auch noch einen weiteren Anwendungsfall, der damit nur schlecht berücksichtigt werden kann: *Art Direction*. Dieser Begriff bezeichnet den Umstand, dass bei verschiedenen Auflösungen nicht nur unterschiedliche Größen von Bildern zum Einsatz kommen, sondern diese sich auch inhaltlich unterscheiden können. So können bei breiten Anzeigen entsprechend breite Bilder verwendet werden, bei schmalen Viewports (z. B. am Handy) kann eine umgestaltete, schmale Version zum Einsatz kommen.

Dies kann mit dem **<picture>**-Element erreicht werden. Dieses ist ein Container für verschiedene Versionen, diese werden mit mehreren **<source>** Elementen angegeben. Über das Attribut **media** kann das Kriterium angegeben werden, wann dieses Element zum Einsatz kommt, **srcset** spezifiziert die zu verwendende Datei. Die Optionen im Bezug auf Auflösung und Pixeldichte können ebenso wieder angegeben werden.

Im folgenden Beispiel wird ab einer Breite von 990 Pixeln ein HTML5 Sticker angezeigt. Für alle schmälere Viewports kommt das reguläre HTML5 Logo zum Einsatz.

```
1 <picture>
2   <source media="(min-width: 990px)" srcset="html5sticker.png 1800w">
3   
4 </picture>
```

Einsatz verschiedener Grafikformate

Ebenso kann `<picture>` verwendet werden, um den Browser aus mehreren *Grafikformaten* wählen zu lassen. So kann mit dem `type`-Attribut etwa eine Vektorgrafik und eine Pixelgrafik angegeben werden. Unterstützt der Browser erstere, wird sie angezeigt, ansonsten auf die Pixelgrafik ausgewichen. Das folgende Markup stellt dies dar:

```
1 <picture>
2   <source srcset="html5logo.svg" type="image/svg+xml">
3   
4 </picture>
```

2.5.4 Abbildungen

In traditionellen Printpublikationen werden Bilder in der Regel mit Beschreibungen (engl. „captions“) versehen. Ein Beispiel sind die Abbildungen in diesem Skriptum. Seit HTML5 ist dies auch für Webseiten möglich. Die dafür verantwortlichen Elemente sind `<figure>` (mit dem abschließenden `</figure>`) und `<figcaption>` (mit `</figcaption>` als Abschluss).

`<figure>` wird verwendet, um eine Abbildung einzuleiten. Alles innerhalb dieses Elements gehört zu dieser Abbildung. In der Regel wird hier ein `<img>`-Element stehen, aber auch Text wie z. B. Quellcode kann in einer Abbildung platziert werden. Um der dieser eine Beschreibung zu geben, wird ebenfalls noch innerhalb das `<figcaption>`-Element platziert. Wichtig: Diese Beschreibung ersetzt keinesfalls das `alt`-Attribut beim Bild selbst.

Es folgt das Beispiel von vorher, jedoch mit einer Beschreibung in einer Abbildung platziert. Das Ergebnis ist in Abbildung 2.8 ersichtlich.

```
1 <figure>
2   
3   <figcaption>Das offizielle HTML5-Logo. Quelle: <a href="https://www.w3.org/html/
   logo/">W3C</a></figcaption>
4 </figure>
```

Bildformate im Web

Im Web haben sich mittlerweile fünf Bildformate für Pixelbilder etabliert, die von allen modernen Browsern unterstützt werden: GIF, PNG, JPEG, WebP und AVIF. Letzteres funktioniert seit Ende Jänner 2024 allen wichtigen Browsern. Neben Pixelbildern unterstützen moderne Browser auch Vektorbilder. Im Web hat sich dabei SVG als Format durchgesetzt.

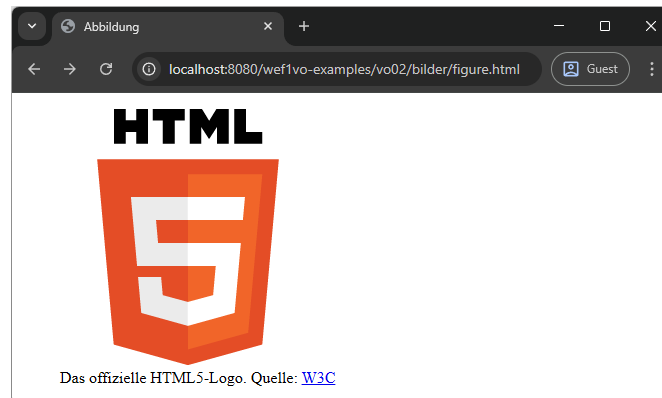


Abbildung 2.8: Die Bilddatei, verpackt in einer Abbildung mit beschreibendem Text.

GIF: Das *Graphics Interchange Format* mit der Dateiendung **.gif** ist ein verlustfrei komprimiertes Bildformat, das 256 Farben darstellen kann. Es unterstützt Transparenzen, Animationen und Interlacing (schichtweisen Aufbau). GIF eignet sich gut für kleine Grafiken wie Buttons oder simple Farbelemente ohne Verläufe.

PNG: Das *Portable Network Graphics* Format mit der Endung **.png** wurde speziell für den Einsatz im Web konzipiert. Es unterstützt 16,7 Millionen Farben und komprimiert ebenfalls verlustfrei. Darüber hinaus sind Transparenz und Interlacing möglich. PNG ist der moderne Nachfolger von GIF, der für die selben Einsatzzwecke gedacht und mit der höheren Farbtiefe auch flexibler ist.

JPEG: JPEG steht für *Joint Photographic Expert Group* und bezeichnet somit das Konsortium, das das Format entwickelt hat. JPEG (Dateiendungen **.jpg** oder auch **.jpeg**) verwendet ein verlustbehaftetes Kompressionsverfahren und unterstützt 16,7 Millionen Farben. JPEG ist sehr gut für Fotografien geeignet, mit dem Kompressionsfaktor lässt sich die Qualität des Bildes in Abhängigkeit von der Dateigröße steuern. Transparenzen werden nicht unterstützt, Interlacing liegt im so genannten Progressive-Verfahren vor.

WebP: WebP [4] – ausgesprochen als *weppy* – ist ein von Google aus dem VP8-Videocodec entwickeltes Format. Es unterstützt sowohl verlustbehaftete als auch verlustfreie Kompression. Bei erster werden laut eigenen Angaben bessere Bildqualitäten bei gleicher Dateigröße als bei JPEG erzielt. WebP wird in allen aktuellen Browsern unterstützt, auch Adobe Photoshop unterstützt das Format nativ. Die Dateiendung lautet **.webp**. Da WebP den anderen drei Formaten in quasi allen Facetten überlegen ist, sollte es mittlerweile als das Standard-Format für Pixelbilder im Web angesehen werden.

AVIF: Das AV1 Image File Format (AVIF) [1] – Dateiendung **.avif** – ist ein offenes Grafikformat für Rastergrafiken, basierend auf dem AV1 Videocodec. Es kann bis zu 25 % kleinere Dateien als WebP in der verlustbehafteten Kompressionsmethode produzieren. Weiters existiert auch eine verlustfreie Version. Während es kleinere Dateien als WebP produziert, benötigen diese oft etwas länger zum Dekodieren, was die Ladezeiten nicht immer dramatisch erhöht.

SVG: *Scalable Vector Graphics* [12] (mit der Endung **.svg**) ist ein zweidimensionales Vektorformat, das auf der Beschreibungssprache XML (siehe Vorlesung 7) basiert. Die Unterstützung für SVG-Bilder im Browser ist seit Jahren sehr gut. Alle wichtigen Browser ermöglichen das Einbinden von SVG-Bildern mittels ****-Tag, auch als CSS-Hintergründe funktionieren SVG-Bilder bereits zuverlässig.

2.5.5 Lazy Loading von Bildern

Wird ein Bild mit dem `<img>`-Tag eingebunden, so fordert der Browser diese externe Ressource (das Bild) sofort an, d. h. er lädt es sofort herunter und zeigt es alsbald an. Dieses Standardverhalten nennt man auch *eager Loading* und ist für Elemente, die sofort auf der Seite sichtbar sein sollen, gewünscht, um Verzögerungen zu vermeiden.

Oft gibt es jedoch Bilder, die erst später, d. h. weiter unten auf einer Seite, benötigt werden. Diese werden erst angezeigt, wenn der*die User*in auch in diesen Bereich scrollt. Da nicht sicher ist, ob dies überhaupt passiert, ist eager Loading hier unter Umständen kontraproduktiv, da Bilder geladen und übertragen werden, die nie für den*die User*in sichtbar werden.

Aus diesem Grund kann das Attribut `loading` mit dem Wert `lazy` beim `<img>`-Tag verwendet werden. In diesem Fall beginnt der Browser erst mit dem Laden des Bildes, wenn der*die User*in in dessen Nähe scrollt.

Das Markup dazu sieht wie folgt aus:

```
1 
```

Achtung: Lazy Loading sollte nicht für Bilder verwendet werden, die sofort im Viewport sichtbar, d. h. am Beginn der Seite, sind. Ebenso muss JavaScript aktiviert sein, damit lazy Loading funktioniert.

2.6 Formulare

Formulare sind interaktive Komponenten einer Webpage, die es Benutzer*innen ermöglichen, Daten aktiv einzugeben. Ein Formular besteht aus verschiedenen Formularelementen (engl. „form controls“) wie z. B. Textfelder, Buttons, Checkboxes oder Slider. Die Erstellung eines Formulars umfasst zunächst die Erstellung des User-Interfaces, in dem eben jene Elemente platziert werden und in weiterer Folge die Weiterverarbeitung der eingegebenen Daten. Diese Weiterverarbeitung geschieht meist serverseitig, Formulardaten können jedoch auch ganz einfach in ein E-Mail verpackt werden.

2.6.1 Ein Formular erstellen

Jedes Formular beginnt mit dem `<form>`-Element und wird mit `</form>` abgeschlossen. Alle Formularelemente werden innerhalb davon platziert.

`<form>` besitzt eine Reihe von Attributen, die bestimmen, auf welche Art die eingegebenen Daten weiterverarbeitet werden. Die wichtigsten sind im Folgenden aufgelistet, die vollständige Liste findet sich in [9, Abschnitt 4.10.3].

- **action:** Gibt das Ziel des Formulars an. Kann als Wert einen URL beinhalten oder auch die Angabe einer E-Mail-Adresse mit `mailto:.` Beim Absenden des Formulars wird dieses Ziel dann aufgerufen.
- **method:** Gibt die Übertragungsmethode an. Zur Auswahl stehen die Werte `get` und `post`. Diese Unterscheidung ist derzeit noch nebensächlich, als Methode kann einfach `post` verwendet werden. Details dazu folgen im zweiten Semester, wenn es um serverseitige Weiterverarbeitung von Formularen geht.
- **enctype:** Gibt die Kodierung der eingegebenen Daten zur Übertragung in Form eines so genannten MIME-Types an. Es sind drei verschiedene Werte vorgesehen:

`application/x-www-form-urlencoded`, `multipart/form-data` und `text/plain`. Ersteres ist der Standardwert, der die Daten in einer zusammengehängten URL-Form weitergibt. Der zweite Typ ist beim Verarbeiten von im Formular hochgeladenen Dateien wichtig. Der letzte Typ gibt Daten als Klartext lesbar aus. Diese Option sollte beim E-Mail-Versand gewählt werden.

- **name**: Der Name des Formularfelds. Dieser kann optional angegeben werden und ist meist nur dann sinnvoll, wenn auf das Formular mittels einer Skriptsprache wie JavaScript oder PHP zugegriffen werden soll.
- **autocomplete**: Dieses Attribut bestimmt, ob die browserinterne Autovervollständigung aller Eingabefelder im Formular (Liste mit vorgeschlagenen Wörtern, die bereits einmal eingegeben wurden) aktiviert ist, oder nicht. Es kann die Werte `on` oder `off` haben, wobei ersteres nicht angegeben werden muss, da die Autovervollständigung standardmäßig aktiviert ist.
- **novalidate**: Dieses Standalone-Attribut (ohne Wertangabe in Anführungszeichen) gibt an, dass das Formular vom Browser nicht auf Gültigkeit überprüft werden soll. Mehr dazu in Abschnitt 2.6.8.

Das Grundgerüst für ein Formular, das per E-Mail übertragen werden soll, sieht wie folgt aus. Es alleine besitzt noch keine visuelle Repräsentation sondern markiert nur einen Bereich als Formular.

```
1 <form action="mailto:john.doe@johndoe.com" method="post" enctype="text/plain">
2   ...
3 </form>
```

2.6.2 Formularfelder beschriften

Bevor die verschiedenen Elemente zur Dateneingabe – die eigentlichen Formularfelder – besprochen werden, sei erwähnt, dass jedes solche Feld auch eine Beschriftung benötigt. Während ein neben ein Eingabefeld platzierter Absatz natürlich visuell ausreichen würde, gibt es ein eigens dafür bestimmtes Element, das auch eine semantische Zuordnung zwischen *Beschriftung* und Eingabefeld ermöglicht: `<label>`.

Um die Zuordnung eindeutig zu ermöglichen, besitzt das Element ein Attribut `for`, dessen Wert mit dem `id`-Attribut des Eingabefeldes übereinstimmen muss. `id` ist ein Universalattribut, das einem Element einen eindeutigen Namen gibt – ideal für die eindeutige Zuordnung einer Beschriftung. Die meisten Browser setzen beim Klick auf die Beschriftung den Cursor in das zugeordnete Eingabefeld.

In den nachfolgenden Beispielen zu den verschiedenen Formularfeldern ist jeweils bereits eine Beschriftung beigelegt.

2.6.3 Eingabefelder

Eingabefelder (engl. „input fields“) sind die zentralen Elemente zur Dateneingabe in Formularen. Es handelt sich dabei um ein einzeiliges, typisiertes Datenfeld, d.h. dasselbe HTML-Element, nämlich `<input>` wird für eine Vielzahl an verschiedenen Anwendungsfällen verwendet und durch das `type`-Attribut näher bestimmt. Es handelt sich dabei um ein Standalone-Element. Eingabefelder müssen innerhalb des `<form>`-Elements platziert werden.



Abbildung 2.9: Zwei identische Textfelder. Die Angabe von `type="text"` ist in diesem Fall also optional. Eingabefelder und Label sind Phrasing Content und erzeugen keine neue Zeile pro Element, der Zeilenumbruch wurde mittels `<br>`-Tag herbeigeführt. In das erste Feld wurde zur Veranschaulichung Text eingegeben, die Beschriftung erfolgte in beiden Fällen mittels `<label>`-Element.

Es gibt eine Reihe von Attributen, die bei den meisten Feldern anwendbar sind. Einige wichtige sind im Folgenden aufgelistet, weitere werden im Verlauf des Abschnitts noch diskutiert, eine komplette Übersicht bietet der HTML Living Standard [9, Abschnitt 4.10.5].

- **value:** Erlaubt es, den Wert eines Eingabefeldes bereits vorab zu definieren (vor auszufüllen). Bei Checkboxes und Radiobuttons kann mit **value** angegeben werden, welcher Wert übertragen werden soll, wenn die Checkbox ausgewählt wurde.
- **size:** Bestimmt die Länge eines Eingabefeldes, also wie lange es optisch aussieht. Die Zahl entspricht der Zeichenanzahl, die im Formularfeld Platz findet, bevor der Text horizontal innerhalb des Feldes zu scrollen beginnt.
- **maxlength:** Der Wert dieses Feldes bestimmt, wie viel Zeichen tatsächlich in das Eingabefeld geschrieben werden können. Diese Angabe muss nicht mit **size** übereinstimmen.
- **readonly:** Dieses Standalone-Element verhindert, dass mit einem Formularelement interagiert werden kann. Readonly-Elemente werden von Browsern in der Regel ausgegraut angezeigt.

Im Folgenden werden exemplarisch drei Varianten (**text**, **password** und **email**) des einzeiligen Eingabefeldes besprochen. Weitere Typen sind **hidden**, **search**, **tel**, **url**, **date**, **month**, **week**, **time**, **datetime-local**, **number**, **range**, **color** und **file**. Details zu diesen Typen finden sich in [9, Abschnitt 4.10.5].

Text

Textfelder werden mit dem Wert **text** für das **type**-Attribut des `<input>`-Elements erzeugt. In diesem Fall kann das Attribut aber auch ganz entfallen, da **text** der Standardwert für Eingabefelder ist, wenn nichts angegeben wird.

Um ein Textfeld zu erzeugen, führen somit sowohl das folgende Markup in Zeile 2 als auch das in Zeile 4 zum (selben) Ziel. Das Ergebnis ist in Abbildung 2.9 zu sehen.

```
1 <label for="with">Input mit Attribut:</label>
2 <input type="text" id="with"><br>
3 <label for="without">Input ohne Attribut:</label>
4 <input id="without">
```

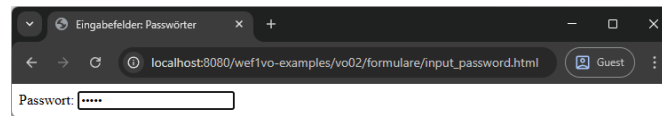


Abbildung 2.10: Ein Passwort-Eingabefeld. Eingebener Text wird vom Browser maskiert.

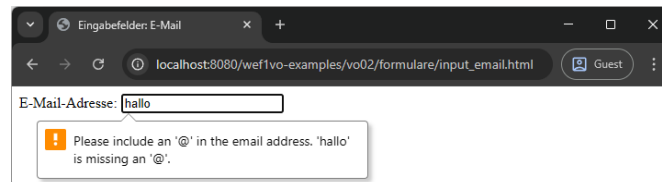


Abbildung 2.11: Ein Eingabefeld vom Typ E-Mail in Verwendung. Im Browser wird bei Eingabe falscher Daten (keine E-Mail-Adresse) das Absenden des Formulars verhindert.

Passwörter

Passwortfelder haben den Typ `password`. Sie funktionieren wie Textfelder, die eingegebenen Zeichen werden jedoch vom Browser nicht angezeigt, sondern durch Platzhalter ersetzt.

Das Markup für ein Passwortfeld sieht wie folgt aus, das Ergebnis (mit einem eingetippten Wort) ist in Abbildung 2.10 zu sehen.

```
1 <label for="pass">Passwort:</label>
2 <input type="password" id="pass">
```

E-Mail-Adressen

Der Formulartyp für *E-Mail-Adressen* unterscheidet sich auf einem Desktop-Browser meist visuell nicht von einem Textfeld, bringt jedoch andere Vorteile mit. So führen alle modernen Browser beim Absenden des Formulars eine inhaltliche Überprüfung für Felder vom Typ E-Mail durch. Wird ein Wort eingegeben, dessen Format keiner E-Mail-Adresse entspricht, so verhindert der Browser das Abschicken des Formulars und gibt eine Fehlermeldung aus (Abbildung 2.11). Auf Geräten mit virtueller Tastatur, wie etwa Smartphones, bewirkt das Auswählen eines E-Mail-Feldes eine Umstellung der Tastatur. So werden die für E-Mail-Adressen relevanten Zeichen „@“ und „.“ auf dem Primärkeyboard eingeblendet).

Wird das Attribut `multiple` angegeben, erlaubt das Feld die Angabe von mehreren E-Mail Adressen, die mit Komma getrennt sind. Dies kann nützlich sein, wenn etwa eine Benutzeroberfläche für ein Webmail Programm erstellt wird. Die „To“, „CC“ oder „BCC“ Felder könnten dann mehrere Adressen erlauben.

Das Markup für ein einfaches E-Mail Feld sieht wie folgt aus:

```
1 <label for="e-mail">E-Mail-Adresse:</label>
2 <input type="email" id="e-mail">
```

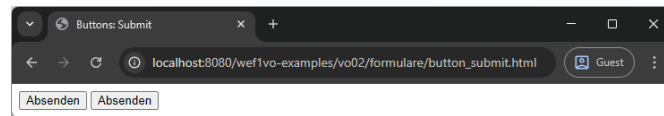


Abbildung 2.12: Zwei Submit-Buttons mit derselben Funktionalität und demselben Aussehen – einmal mittels `<button>`, das andere Mal mittels `<input>`-Element.

Autofill bei E-Mail-Feldern: Durch verschiedene Werte beim `autocomplete`-Attribut (anstelle von `on` und `off`) ist für den Browser eine Unterscheidung möglich, welche Inhalte er bei einem Formularfeld vorschlägt. So schlägt etwa `autocomplete="work email"` die berufliche E-Mail-Adresse vor, `autocomplete="home email"` eine private. Der HTML-Standard definiert hier eine Reihe von Werten, mit denen dem Browser mitgeteilt werden kann, welche Inhalte erwartet werden. Diese können in [9, Abschnitt 4.10.19.7.1] eingesehen werden.

Somit kann ein Eingabefeld für eine E-Mail-Adresse wie folgt gestaltet sein:

```
1 <input type="email" autocomplete="home email">
```

2.6.4 Buttons

Buttons haben die Hauptaufgabe, Formulare abzusenden. Während bei einzeiligen Eingabefeldern ein Absenden durch Drücken der Enter-Taste möglich ist, sollte auf Buttons nie verzichtet werden. Zusätzlich zum Absenden (Submit) existiert ein Button zum Zurücksetzen der Formulardaten (Reset), ein allgemeiner Button ohne spezielle Funktion, sowie ein grafischer Button zum Absenden.

HTML bietet zwei unterschiedliche Möglichkeiten, Buttons zu realisieren: Über das `<input>`-Element oder das sprechendere `<button>`-Element. Beide erfüllen denselben Zweck, `<input>` ist jedoch ein Standalone-Element, während `<button>` mit `</button>` abgeschlossen wird und daher Inhalt haben kann. Dieser Inhalt können Text oder auch andere HTML-Elemente sein, was das `<button>`-Element etwas flexibler macht. Beim Element `<input>` kann zur Beschriftung des Buttons lediglich Text mit Hilfe des `value`-Attributs angegeben werden.

Submit-Button

Der *Submit-Button* hat die Aufgabe, ein Formularfeld abzuschicken. Im Folgenden wird das Markup für beide Möglichkeiten angegeben – ein Submit-Button pro `<form>`-Element genügt aber selbstverständlich. Wie schon zuvor bei den einzeiligen Eingabefeldern spezifiziert das `type`-Attribut (in beiden Fällen) die Art des Buttons. Der Wert `submit` steht für den Submit-Button. Hier kann die Angabe entfallen, da `submit` der Standardwert ist. Abbildung 2.12 zeigt die Darstellung beider Buttons.

```
1 <button type="submit">Absenden</button>
2 <input type="submit" value="Absenden">
```

Reset-Button

Der *Reset-Button* setzt die Inhalte eines Formulars auf Standardwerte zurück. In der Regel werden alle Inhalte dadurch gelöscht (es sei denn, es sind Werte über das `value`-Attribut vorgegeben, dann werden diese wiederhergestellt). Der Reset-Button kann wiederum mittels `<input>`- oder `<button>`-Element erzeugt werden, der Wert des `type`-Attributs muss `reset` sein.

Das folgende Markup-Fragment stellt beide Arten dar, ein Reset-Button genügt aber normalerweise. Auf eine Abbildung wird verzichtet, der Button hat standardmäßig dasselbe Aussehen wie ein Submit-Button (nur die Beschriftung wäre anders).

```
1 <button type="reset">Zurücksetzen</button>
2 <input type="reset" value="Zurücksetzen">
```

Allgemeiner Button

Neben Submit und Reset existiert noch ein weiterer Button, der keine besondere Funktion besitzt. Wird er angeklickt, so passiert einfach nichts. Dieser Button findet dann Verwendung, wenn nicht mit Formulardaten gearbeitet wird, sondern z. B. über JavaScript bestimmte Funktionalitäten per Klick angestoßen werden sollen. Wiederum existieren zwei Arten, bei beiden muss das `type`-Attribut den Wert `button` haben.

Das folgende Markup zeigt wieder beide Versionen, auf die Abbildung wurde aus zuvor erwähnten Gründen wieder verzichtet.

```
1 <button type="button">Klick mich!</button>
2 <input type="button" value="Klick mich!">
```

Zusätzlich existiert noch ein Button des Typs `image`, der Grafiken als Buttons zulässt.

2.6.5 Mehrzeilige Eingabefelder

Während einzeilige Eingabefelder durch das `<input>`-Element repräsentiert werden, werden *mehrzeilige Felder* mit dem `<textarea>`-Element ausgezeichnet. Es handelt sich dabei um ein mehrzeiliges Textfeld, andere Typen sind nicht vorhanden. Auch gibt es ein schließendes Element `</textarea>`. Somit wird ein eventuell vorbelegter Text nicht mit dem `value`-Attribut notiert, sondern einfach zwischen öffnendes und schließendes Element gesetzt. In mehrzeiligen Textfeldern bewirkt das Drücken der Enter-Taste einen Zeilenumbruch und nicht das Absenden des Formulars.

Um die Größe des Feldes festzulegen, gibt es die Attribute `rows` (Anzahl der Zeilen) und `cols` (Anzahl der Spalten bzw. Zeichen). Mittels `maxlength` kann die maximal erlaubte Zeichenlänge festgelegt werden. Das Attribut `readonly` setzt das Feld auf nur lesbar. Das Attribut `wrap` kann die Werte `soft` und `hard` annehmen. Der Standardzustand `soft` bewirkt, dass nur diese Zeilenumbrüche übertragen werden, die explizit durch Drücken von Enter gesetzt wurden. Wurde `hard` angegeben, so werden jene Zeilenumbrüche übertragen, wie sie durch die Größe des Eingabefeldes zustande kamen. Diese muss dann aber zwingend über das `cols`-Attribut spezifiziert werden.

Abbildung 2.13 zeigt ein Formularfeld, das mit dem folgenden HTML-Markup erstellt wurde:

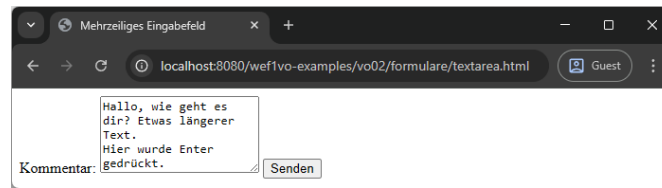


Abbildung 2.13: Ein mehrzeiliges Eingabefeld, dessen Höhe auf fünf Zeilen gesetzt wurde und das mit längerem Fließtext befüllt wurde.

```
1 <label for="comment">Kommentar:</label>
2 <textarea name="text" rows="5" id="comment"></textarea>
3 <button type="submit">Senden</button>
```

In diesem Beispiel wurde das `wrap`-Attribut nicht gesetzt, also wird `soft` als Wert angenommen. Die Ausgabe in einem E-Mail sieht wie folgt aus (`text=` rührt vom `name`-Attribut her, das mit dem Wert „text“ gesetzt wurde):

```
text=Hallo, wie geht es dir? Etwas längerer Text.
Hier wurde Enter gedrückt.
```

Wird nun die Attributliste um `wrap="hard"` (und die Breitenangabe `cols="20"`) ergänzt, sieht die Ausgabe wie folgt aus. Die Umbrüche sind genau so, wie sie durch die Breite des Textfeldes entstanden sind. Dies deckt sich mit Abbildung 2.13.

```
text=Hallo, wie geht es
dir? Etwas längerer
Text.
Hier wurde Enter
gedrückt.
```

2.6.6 Radio Buttons und Checkboxes

Radio Buttons und Checkboxes erlauben es, Auswahlen in einem Formular zu treffen. Radio Buttons bieten dabei n Optionen und erlauben genau 1 Auswahl daraus, Checkboxes bieten ebenfalls n Optionen und erlauben m Auswahlen, d.h. es können alle Kombinationen angehakt werden, oder auch gar nichts.

Radio Buttons

Radio Buttons werden mit dem `<input>`-Element erstellt, das `type`-Attribut muss dabei den Wert `radio` haben. Um eine Gruppe von Radio Buttons zu erzeugen, müssen mehrere solche Elemente im selben Formular erzeugt werden, deren `name`-Attribut sich gleicht. Dadurch wird Zusammengehörigkeit erlangt. Um einen Radio Button aus dieser Gruppe als vorausgewählt zu kennzeichnen, kann diesem das Attribut `checked` gesetzt werden. Soll beim Absenden des Formulars ein bestimmter Wert übergeben werden, kann dieser im `value`-Attribut angegeben werden.

Das folgende Markup zeigt eine Auswahl aus drei Elementen, wobei das erste vorausgewählt ist. Die Abbildung 2.14 zeigt die Darstellung im Browser.

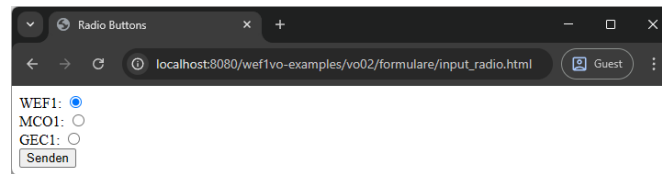


Abbildung 2.14: Drei Radio Buttons mit Beschriftung – der erste ist vorausgewählt.

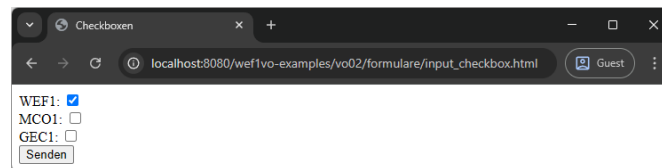


Abbildung 2.15: Drei Checkboxes mit Beschriftung – die erste ist vorausgewählt.

```

1 <label for="wef">WEF1:</label>
2 <input type="radio" name="favorit" value="WEF1" id="wef" checked><br>
3 <label for="mco">MCO1:</label>
4 <input type="radio" name="favorit" value="MCO1" id="mco"><br>
5 <label for="gec">GEC1:</label>
6 <input type="radio" name="favorit" value="GEC1" id="gec"><br>
7 <button type="submit">Senden</button>

```

Checkboxen

Checkboxen werden ebenfalls mit dem `<input>`-Element erstellt, das `type`-Attribut muss dabei den Wert `checkbox` haben. Checkboxes müssen nicht als zusammengehörig markiert werden, d. h. das `name`-Attribut muss nicht angeführt werden bzw. übereinstimmen. Die Attribute `value` und `checked` funktionieren gleich wie bei Radio Buttons.

Das folgende Markup zeigt das obige Beispiel mit Checkboxes realisiert, Abbildung 2.15 zeigt das Ergebnis im Browser.

```

1 <label for="wef">WEF1:</label>
2 <input type="checkbox" value="WEF1" id="wef" checked><br>
3 <label for="mco">MCO1:</label>
4 <input type="checkbox" value="MCO1" id="mco"><br>
5 <label for="gec">GEC1:</label>
6 <input type="checkbox" value="GEC1" id="gec"><br>
7 <button type="submit">Senden</button>

```

2.6.7 Auswahllisten

Auswahllisten, auch Dropdown-Listen genannt, ermöglichen die Auswahl aus einer Liste von Optionen. Um eine solche Liste zu erstellen, wird das Element `<select>` (geschlossen durch `</select>`) verwendet. Mit dem Attribut `name` kann, wie bei Eingabefeldern, ein Name für die Liste vergeben werden. `size` gibt an, wie viele Optionen auf einmal gezeigt werden sollen (Standardwert ist 1). Über die Angabe von `multiple` können durch Drücken von `Strg` (Windows) oder `⌘` (Mac) mehrere Einträge ausgewählt werden.

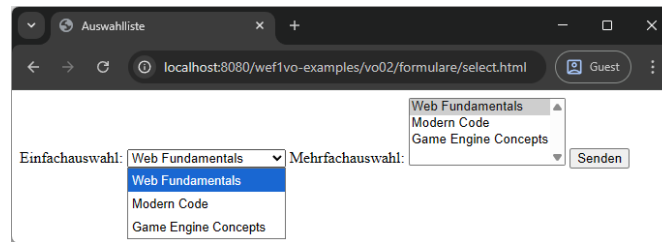


Abbildung 2.16: Zwei Mal dieselbe Auswahlliste, links mit Einfachauswahl, rechts mit Mehrfachauswahl.

Die Einträge werden durch `<option>`-Elemente (abgeschlossen durch `</option>`) innerhalb des `<select>`-Elements erstellt. Über das Attribut `value` kann der Wert zur Übertragung bestimmt werden (wenn dieser vom angezeigten abweichen soll), `selected` spezifiziert den Eintrag, der vorausgewählt sein soll. Um Einträge zu gruppieren, können mehrere `<option>`-Elemente in `<optgroup>`-Tags verpackt werden.

Das folgende Markup stellt zwei Auswahllisten mit drei Elementen dar, einmal mit Einfach-, dann mit Mehrfachauswahl. Abbildung 2.16 zeigt die Darstellung im Browser.

```

1 <label for="einfach">Einfachauswahl:</label>
2 <select name="faecher1" id="einfach">
3   <option value="WEF1" selected>Web Fundamentals</option>
4   <option value="MCO1">Modern Code</option>
5   <option value="GEC1">Game Engine Concepts</option>
6 </select>
7 <label for="mehrfach">Mehrfachauswahl:</label>
8 <select name="faecher2" id="mehrfach" multiple>
9   <option value="WEF1" selected>Web Fundamentals</option>
10  <option value="MCO1">Modern Code</option>
11  <option value="GEC1">Game Engine Concepts</option>
12 </select>
13 <button type="submit">Senden</button>

```

Auswahllisten stylen

Seit Jänner 2025 gibt es das neue Element `<selectedcontent>`. Dieses wird innerhalb eines Buttons noch vor den `<option>`-Elementen eingefügt und enthält immer die aktuell ausgewählte Option. Wird das Element gesetzt und die Auswahlliste mit dem folgenden CSS-Code formatiert, so wird die Standardformatierung des Browsers bzw. des Betriebssystems deaktiviert und die Auswahlliste kann komplett angepasst werden. So ist es etwa auch möglich, HTML innerhalb der `<option>`-Elemente zu verwenden, Bilder einzufügen und ähnliches.

```

1 <label for="customized">Einfachauswahl angepasst:</label>
2 <select name="faecher3" id="customized">
3   <button><selectedcontent></selectedcontent></button>
4   <option value="WEF1" selected>Web Fundamentals <strong>WEF1</strong></option>
5   <option value="MCO1">Modern Code <strong>MCO1</strong></option>
6   <option value="GEC1">Game Engine Concepts <strong>GEC1</strong></option>
7 </select>
8 <button type="submit">Senden</button>

```

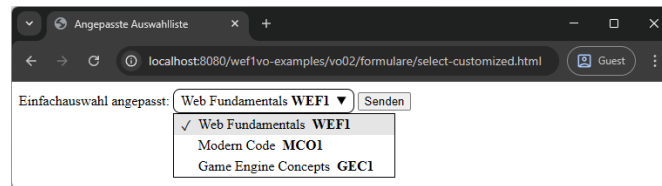


Abbildung 2.17: Eine angepasste Auswahlliste. Die Abkürzung der Lehrveranstaltung erscheint fett, weil sie mit `<strong>` formatiert ist.

```
1 select,
2 ::picker(select) {
3   appearance: base-select;
4 }
```

Das Feature ist bereits teil des HTML Living Standards, wird aber zum aktuellen Zeitpunkt nur von den Blink-basierten Browsern (Chrome, Edge, Opera) unterstützt. Abbildung 2.17 zeigt die Einfachauswahlliste mit einem durch `<strong>` fett gesetzten Wort.

2.6.8 Zusatzangaben bei Formularelementen

Bei den meisten Formularfeldern gibt es drei interessante Zusatzangaben, die hinzugefügt werden können, um kleine, aber feine Verbesserungen in der Benutzbarkeit zu erlangen: Platzhaltertext, Autofokus und Pflichtfelder.

Platzhaltertext

Auch wenn über die Beschriftung mit `<label>` angegeben werden kann, was in ein Formularfeld eingegeben werden soll (z. B. Name oder E-Mail-Adresse), so ist es oft praktisch, zusätzliche Informationen angeben zu können, wie etwa ein Beispieleintrag aussehen könnte. Genau diese Sache kann über *Platzhaltertext* gut gelöst werden. Während das Formularfeld unausgefüllt ist, wird der Platzhaltertext angezeigt, sobald das Feld ausgewählt bzw. mit Inhalt befüllt wurde, verschwindet dieser. Verantwortlich dafür ist das Attribut `placeholder`. Sein Wert wird im Formularfeld angezeigt. Dieses Attribut darf nicht mit `value` verwechselt werden. Denn während `value` wirklich den Wert vorausfüllt (d. h. beim Absenden würde der Wert des `value`-Attributs auch übermittelt werden), wird der Platzhaltertext nur visuell angezeigt und beim Absenden gilt das Formular als leer.

Das folgende Markup zeigt ein E-Mail Feld mit Platzhaltertext, Abbildung 2.18 stellt das Ergebnis im Browser dar.

```
1 <label for="contact">E-Mail:</label>
2 <input type="email" id="contact" placeholder="user@example.com">
3 <button type="submit">Absenden</button>
```

Autofokus

Websites, deren Haupteinsatzgebiet die Suche nach Information ist (z. B. Suchmaschine, Enzyklopädie, Wörterbuch etc.), sollen den Benutzer*innen einen möglichst effektiven

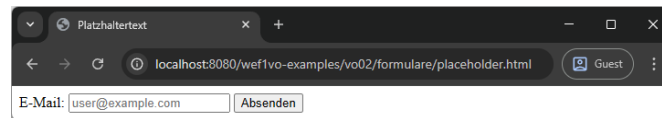


Abbildung 2.18: Der Platzhaltertext wird im Eingabefeld angezeigt. Klickt der*die Benutzer*in hinein, ist das Feld leer.

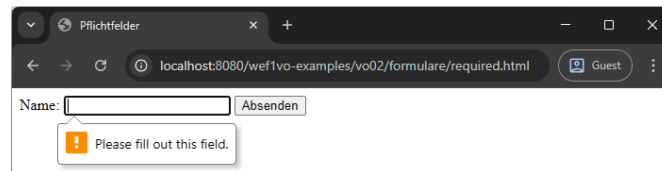


Abbildung 2.19: Wird das Feld nicht ausgefüllt, zeigt der Browser eine Fehlermeldung und verhindert das Absenden.

Ablauf bieten. Meist wird die Seite aufgerufen, der Suchbegriff eingegeben und die Suche abgeschickt. Um diesen Ablauf zu optimieren, bietet es sich an, dem zentralen Suchfeld das Attribut `autofocus` anzufügen. Somit steht der Cursor nach dem Aufruf der Seite sofort in diesem Feld und Benutzer*innen müssen es nicht zuerst mit Mouse, Keyboard oder durch Antippen anwählen.

Das folgende Markup statet ein Suchfeld mit der *Autofokus*-Eigenschaft aus. Auf eine Abbildung wird verzichtet, es wird empfohlen dieses Verhalten mit einer Beispieldatei auszuprobieren.

```
1 <label for="search-box">Suche:</label>
2 <input type="search" id="search-box" autofocus>
3 <button type="submit">Absenden</button>
```

Pflichtfelder

Wie bereits beim Eingabefeld für E-Mail-Adressen (Abschnitt 2.6.3) erwähnt, sieht der HTML-Standard Validierung von Eingaben durch den Browser vor (wo es, aufgrund eindeutiger Muster möglich ist). Eine weitere Möglichkeit besteht darin, Eingabefelder als *Pflichtfelder* zu markieren. Werden sie nicht ausgefüllt, verhindert der Browser das Absenden des Formulars. Dies ist durch Angabe des Attributs `required` möglich.

Nachfolgend das Markup für ein verpflichtendes Textfeld und die Warnung des Browsers in Abbildung 2.19.

```
1 <label for="name">Name:</label>
2 <input type="text" id="name" required>
```

2.6.9 Formulare strukturieren

Bei größeren Formularen ist oft eine logische Gruppierung erwünscht, um mehr visuelle Klarheit zu schaffen. So könnten in einem Bestellformular etwa persönliche Daten gruppiert und dann die Angaben zur Bestellung wiederum separat gegliedert werden. Diese

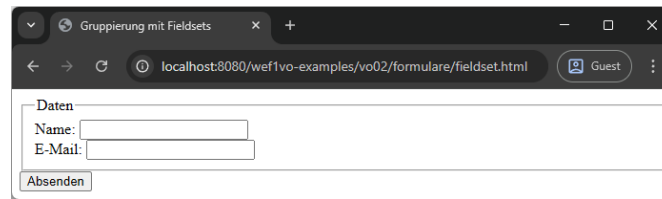


Abbildung 2.20: Die beiden Eingabefelder sind logisch durch ein Fieldset gruppiert. Dieses ist zusätzlich beschriftet.

Gruppierung lässt sich über so genannte *Fieldsets* lösen, das Element hierfür lautet daher `<fieldset>` (mit abschließendem `</fieldset>`) – es enthält alle Formularelemente, die zu einer Gruppe zusammengefasst werden sollen. Visuell wird durch die Angabe eines Fieldsets ein Rahmen um diese Elemente gezogen. Um dem Fieldset eine Beschriftung zu geben, kann das Element `<legend>` (mit `</legend>` als schließendes Element) gleich als erster Tag innerhalb der Gruppierung angegeben werden.

Das folgende Markupbeispiel enthält zwei Eingabefelder, die mit einem beschrifteten Fieldset gruppiert wurden. Abbildung 2.20 zeigt die Darstellung im Browser.

```

1 <fieldset>
2   <legend>Daten</legend>
3   <label for="name">Name:</label>
4   <input type="text" id="name"><br>
5   <label for="contact">E-Mail:</label>
6   <input type="email" id="contact">
7 </fieldset>
8 <button type="submit">Absenden</button>

```

2.6.10 Vollständiges HTML-Grundgerüst mit Formular

Mit dem Wissen über Zeichenkodierungen aus Abschnitt 2.1 lässt sich das ursprünglich in Abschnitt 1.5.6 vorgestellte Grundgerüst nun um die Angabe der Zeichenkodierung ergänzen. Als solche kommt in modernen Websites nur noch UTF-8 in Frage, da sie Probleme mit Sonderzeichen aus verschiedenen Regionen endgültig eliminiert.

Nochmals explizit erwähnt sei die Angabe des `lang`-Attributs beim `<html>`-Element. Dieses Attribut gehört zu den globalen Attributen, kann also bei jedem Element angefügt werden. Durch die Angabe beim Root-Element wird die Sprache des Dokuments bekannt gegeben. Dies ist hilfreich für Suchmaschinen oder andere Tools wie Screenreader für Personen mit visuellen Einschränkungen.

Als Werte können Angaben im Format von Standard-Sprachtags wie z. B. `de` für Deutsch oder `en` für Englisch gemacht werden. Zusätzlich kann optional eine Region mit Bindestrich angehängt und somit noch eine genauere Spezifizierung vorgenommen werden. `de-DE` steht etwa für Deutsch (Deutschland), `de-AT` für Deutsch (Österreich), `en-US` für Englisch (USA) und `en-GB` für Englisch (Großbritannien). Eine vollständige, wenngleich unübersichtliche Liste bietet die IANA³. Richard Ishida stellt ein praktisches Interface zur Suche⁴ zur Verfügung.

³<https://www.iana.org/assignments/language-subtag-registry/language-subtag-registry>

⁴<https://r12a.github.io/app-subtags/>

Ebenfalls enthalten ist ein Formular mit verschiedenen Formularelementen und Attributen. Das Markup sieht wie folgt aus, das finale Ergebnis ist in Abbildung 2.21 zu sehen.

```

1 <!DOCTYPE html>
2 <html lang="de">
3   <head>
4     <title>Vollständiges HTML-Gründgerüst mit Formular</title>
5     <meta charset="utf-8">
6   </head>
7   <body>
8     <h1>Feedback</h1>
9     <form action="mailto:john.doe@johndoe.com" method="post" enctype="text/plain">
10      <fieldset>
11        <legend>Meine Daten</legend>
12        <label for="autor">Name:</label>
13        <input type="text" name="autor" id="autor" size="20" placeholder="John Doe"
14        required><br>
15        <label for="email">E-Mail:</label>
16        <input type="email" name="email" id="email" size="20" placeholder="
17        user@example.com"><br>
18      </fieldset>
19      <fieldset>
20        <legend>Kommentar und Stimmung</legend>
21        <label for="kommentare">Kommentare:</label>
22        <textarea name="kommentare" id="kommentare" rows="8" cols="20" placeholder="
23        Super Beitrag!" required></textarea><br>
24        <p>Ich bin...</p>
25        <label for="happy">happy</label>
26        <input type="radio" name="stimmung" value="happy" id="happy">
27        <label for="neutral">neutral</label>
28        <input type="radio" name="stimmung" value="neutral" id="neutral">
29        <label for="traurig">traurig</label>
30        <input type="radio" name="stimmung" value="traurig" id="traurig">
31      </fieldset>
32      <button type="submit">Kommentar posten</button>
33      <button type="reset">Zurücksetzen</button>
34    </form>
35  </body>
36</html>

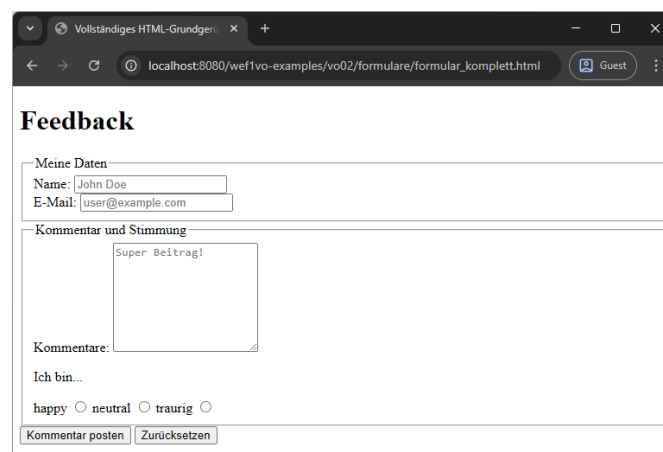
```

2.7 Weiterführende Literatur

Als Nachschlagewerk für HTML-Elemente empfiehlt sich derzeit vor allem der HTML Living Standard der WHATWG [9]. Hier sind selbstverständlich alle Elemente mit allen Details und Regeln aufgelistet.

Eine weitere gute Ressource stellt das Mozilla Developer Network [7] dar. Dieses Wiki kann frei editiert werden und enthält eine sehr kompakte und aktuelle Übersicht zu den Themen HTML, CSS, JavaScript sowie einigen anderen Technologien. Es empfiehlt sich, die englischsprachige Wiki zu verwenden, da die deutsche nicht immer vollständig ist.

Als Informationsquellen können auch [3, 8, 10, 11] dienen, wenngleich die Bücher nicht primäre als Nachschlage-Referenzen gedacht sind und die Elemente ausführlicher erklären.



The screenshot shows a web browser window with the address bar displaying `localhost:8080/wef1vo-examples/vo02/formulare/formular_komplett.html`. The page title is "Vollständiges HTML-Grundgerüst". The main content is a feedback form titled "Feedback".

The form is divided into two sections:

- Meine Daten**: Contains two input fields. The "Name:" field is filled with "John Doe", and the "E-Mail:" field is filled with "user@example.com".
- Kommentar und Stimmung**: Contains a large text area for comments, which has the text "Super Beitrag!". Below the text area, there is a label "Kommentare:" and a line "Ich bin...". Underneath, there are three radio buttons labeled "happy", "neutral", and "traurig", all of which are currently unselected.

At the bottom of the form, there are two buttons: "Kommentar posten" and "Zurücksetzen".

Abbildung 2.21: Ein komplettes HTML-Grundgerüst mit Formular.

Quellenverzeichnis

- [1] Alliance for Open Media. *AV1 Image File Format (AVIF)*. AOM Final Deliverable. Version v1.1.0. 15. Apr. 2022. URL: <https://aomediacodec.github.io/av1-avif/latest-approved.html> (siehe S. 2-21).
- [2] American National Standards Institute. *Coded Character Sets – 7-Bit American Standard Code for Information Interchange (7-Bit ASCII)*. American National Standard for Information Systems. Version ANSI X3.4-1986. 26. März 1986. URL: <https://www.unicode.org/L2/L2006/06388-review-incits4.pdf> (siehe S. 2-3).
- [3] Paul Fuchs. *HTML5 und CSS3 für Einsteiger. Der leichte Weg zur eigenen Webseite*. Landshut, Deutschland: BMU Verlag, 21. Juni 2019 (siehe S. 2-34).
- [4] Google Developers. *WebP. An image format for the Web*. 7. Aug. 2025. URL: <https://developers.google.com/speed/webp/> (besucht am 03. 11. 2025) (siehe S. 2-21).
- [5] International Organization for Standardization. *8-bit single-byte coded graphic character sets – Part 1: Latin alphabet No.1*. ISO/IEC 8859-1:1997. 11. Nov. 1997. URL: <https://www.open-std.org/JTC1/SC2/WG3/docs/n411.pdf> (siehe S. 2-3).
- [6] *meta tags and attributes that Google supports*. Google Search Central. 4. Aug. 2024. URL: <https://developers.google.com/search/docs/crawling-indexing/special-tags> (besucht am 03. 11. 2025) (siehe S. 2-2).
- [7] Mozilla Developer Network and individual contributors. *Mozilla Developer Network*. Nov. 2025. URL: <https://developer.mozilla.org/en-US/> (besucht am 03. 11. 2025) (siehe S. 2-34).
- [8] Peter Müller. *Einstieg in HTML und CSS. Webseiten programmieren und gestalten*. 3. Aufl. Bonn, Deutschland: Rheinwerk Computing, 3. Sep. 2024 (siehe S. 2-34).
- [9] Web Hypertext Application Technology Working Group. *HTML*. Living Standard. 3. Nov. 2025. URL: <https://html.spec.whatwg.org/multipage/> (siehe S. 2-8, 2-12, 2-22, 2-24, 2-26, 2-34).
- [10] Christian Wenz und Christoph Prevezanos. *Jetzt lerne ich HTML5 und CSS3*. 2. Aufl. Burghann, Deutschland: Markt + Technik Verlag, 16. Nov. 2022 (siehe S. 2-34).
- [11] Jürgen Wolf. *HTML und CSS. Das umfassende Handbuch*. 5. Aufl. Bonn, Deutschland: Rheinwerk Computing, 4. Aug. 2023 (siehe S. 2-34).

- [12] World Wide Web Consortium. *Scalable Vector Graphics (SVG) 1.1 (Second Edition)*. W3C Recommendation. 16. Aug. 2011. URL: <https://www.w3.org/TR/SVG11/> (siehe S. 2-21).
- [13] François Yergeau. *UTF-8, a transformation format of ISO 10646*. RFC 3629. IETF, Nov. 2003, S. 1–14. URL: <https://datatracker.ietf.org/doc/rfc3629/> (siehe S. 2-4).